Aakarsh Tyagi — Sec D — MTE Answers

Practical B1: JWT Authentication API (Node.js and Express)

Aim: Implement user registration, login and a protected route using JSON Web Tokens in Express.

Setup: Create a folder, run npm init -y, then install the dependencies below. In package.json add `"type":"module"` and a start script `"start":"node server.js"`.

```
npm install express jsonwebtoken dotenv
```

Create .env with a private secret:

```
JWT_SECRET=replace_with_a_long_random_secret
PORT=3002
```

server.js

```
import express from 'express';
import jwt from 'jsonwebtoken';
import 'dotenv/config';

const app = express();
app.use(express.json());
const users = new Map(); // In-memory users for this practical
const secret = process.env.JWT_SECRET;

app.post('/register', (req, res) => {
  const { username, password } = req.body ?? {};
  if (typeof username !== 'string' || username.trim() === '' ||
      typeof password !== 'string' || password.length < 6) {
    return res.status(400).json({ error: 'Invalid input' });
  }
  const name = username.trim();
  if (users.has(name)) {
    return res.status(409).json({ error: 'Username already exists' });
  }
  users.set(name, password);
  return res.status(201).json({ message: 'User registered' });
});

app.post('/login', (req, res) => {
  const { username, password } = req.body ?? {};
  const name = typeof username === 'string' ? username.trim() : '';
  if (!name || typeof password !== 'string' ||
      !users.has(name) || users.get(name) !== password) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }
  if (!secret) return res.status(500).json({ error: 'Server configuration error' });
  const token = jwt.sign({ username: name }, secret, { expiresIn: '1h' });
  return res.status(200).json({ token });
});

app.get('/profile', (req, res) => {
  const header = req.get('Authorization') || '';
  const match = header.match(/^Bearer\s+(.+)$/i);
  if (!match || !secret) return res.status(401).json({ error: 'Unauthorized' });
  try {
    const payload = jwt.verify(match[1], secret);
    if (typeof payload.username !== 'string')
      return res.status(401).json({ error: 'Unauthorized' });
    return res.status(200).json({ username: payload.username });
  } catch {
    return res.status(401).json({ error: 'Unauthorized' });
  }
});

app.use((err, req, res, next) => {
  if (err instanceof SyntaxError && 'body' in err)
    return res.status(400).json({ error: 'Invalid input' });
  return next(err);
});

const PORT = process.env.PORT || 3002;
app.listen(PORT, () => console.log(`JWT API running on port ${PORT}`));
```

Run and test

```
npm start

# Register
curl -X POST http://localhost:3002/register -H "Content-Type: application/json" -d "{\"username\":\"student1\",\"password\":\"secret1

# Login; copy the returned token
curl -X POST http://localhost:3002/login -H "Content-Type: application/json" -d "{\"username\":\"student1\",\"password\":\"secret123\

# Use the token from the login response
```

```
curl http://localhost:3002/profile -H "Authorization: Bearer TOKEN_HERE"
```

Expected behavior: Register returns 201; invalid input returns 400; duplicate username returns 409. Login returns a one-hour JWT on success or 401 for unknown user/wrong password. Profile returns the username for a valid token and 401 for missing, malformed, invalid-signature or expired tokens.

Security note: This is a classroom example using in-memory storage and plain-text passwords. Real applications should hash passwords, use a database, use HTTPS, and keep the JWT secret private.

Practical B2: Shopping Cart with Discount (React)

Aim: Build a React cart that manages quantities, calculates totals and applies an automatic discount.

Discount assumption: The question does not specify the discount rule. This solution uses 10% off when subtotal is at least 1,000. Change the constants if your instructor specifies another rule.

Setup

```
npm create vite@latest b2-shopping-cart -- --template react
cd b2-shopping-cart
npm install
```

Create src/Cart.jsx

```
import { useState } from 'react';

const DISCOUNT_THRESHOLD = 1000;
const DISCOUNT_RATE = 0.10;
const money = value => Number(value).toFixed(2);

export default function Cart({ products = [] }) {
  const [quantities, setQuantities] = useState({});

  function add(id) {
    setQuantities(current => ({ ...current, [id]: (current[id] || 0) + 1 }));
  }

  function changeQuantity(id, amount) {
    setQuantities(current => {
      const quantity = (current[id] || 0) + amount;
      const next = { ...current };
      if (quantity <= 0) delete next[id];
      else next[id] = quantity;
      return next;
    });
  }

  function remove(id) {
    setQuantities(current => {
      const next = { ...current };
      delete next[id];
      return next;
    });
  }

  const cartItems = products.filter(p => (quantities[p.id] || 0) > 0);
  const total = cartItems.reduce(
    (sum, p) => sum + Number(p.price) * quantities[p.id], 0
  );
  const discount = total >= DISCOUNT_THRESHOLD ? total * DISCOUNT_RATE : 0;
  const grandTotal = total - discount;

  return (
    <main>
      <h1>Shopping Cart</h1>
      <section>
        <h2>Products</h2>
        {products.map(p => (
          <div key={p.id}>
            <span>{p.name} - {money(p.price)}</span>
            <button data-testid={`add-${p.id}`} onClick={() => add(p.id)}>Add</button>
          </div>
        ))}
      </section>
      <section>
        <h2>Your Cart</h2>
        {cartItems.length === 0 ? (
          <p data-testid="cart-empty">Cart is empty</p>
        ) : cartItems.map(p => (
          <div key={p.id}>
            <span>{p.name} - {money(p.price)}</span>
            <button data-testid={`dec-${p.id}`} onClick={() => changeQuantity(p.id, -1)}>–</button>
            <span data-testid={`qty-${p.id}`}>{quantities[p.id]}</span>
            <button data-testid={`inc-${p.id}`} onClick={() => changeQuantity(p.id, 1)}>+</button>
            <button data-testid={`remove-${p.id}`} onClick={() => remove(p.id)}>Remove</button>
          </div>
        ))}
      </section>
      <p data-testid="total">Total: {money(total)}</p>
      {discount > 0 && <>
        <p data-testid="discount">Discount: {money(discount)}</p>
        <p data-testid="grand-total">Grand Total: {money(grandTotal)}</p>
      </>}
    </main>
  );
}
```

Example usage in src/App.jsx

```
import Cart from './Cart.jsx';

const products = [
  { id: 1, name: 'Notebook', price: 170 },
  { id: 2, name: 'Backpack', price: 850 },
  { id: 3, name: 'Pen Set', price: 120 }
];

export default function App() {
  return <Cart products={products} />;
}
```

Run the React app with `npm run dev` and open the local URL shown in the terminal.

How it works: Add adds one item. Increase/decrease updates quantity; when quantity becomes zero the line is removed. Remove deletes a line. Total is formatted to two decimal places. Discount and Grand Total appear only when the threshold is reached.